

```
empl = new SalariedEmployee();
money = empl.calc_pay();
System.out.println( "Money=" + money );
}
}
```

As long as we only call the methods that exist in the Superclass, this will work fine.

```
public class TestEmp
{
    public static void main( String args[] )
    {
        double money = 0.0;
        Employee empl;
        empl = new SalariedEmployee();
        money = empl.calc_pay();
        System.out.println( "Money=" + money );
    }
}
```

So, why in the world might we want to do this?

```
public class RetiredEmployee
extends Employee
{
    double monthlyPension;
    public double calc_pay()
    {
        return monthlyPension;
    }
}
```

Imagine if after we built our system we decided to add a class, for Retired Employees. If we had built our program on a Superclass reference, then we wouldn't have to rebuild anything. The runtime environment sees what kind of object we have instantiated, and calls the right method override.

```
public class TestEmp
{
```